

Requirement on academic honesty (COMP8090SEF/S209W):

1. Each student must make the following declaration on the cover page of the report

Hong Kong Metropolitan University

COMP S209W/8090SEF
Data Structures, Algorithms And Problem Solving
Data Structures, Algorithms

Course Project Report
Group 10

Submission date:

I declare that:

- (i) I have read and checked that all parts of the project (including proposal, code programs, and reports), they are contributed by me, here submitted is original except for source material explicitly acknowledged;
- (ii) the project, in parts or in whole, has not been submitted for more than one purpose without declaration;
- (iii) I am aware of the University's policy and regulations on honesty in academic work and understand the possible consequence when breaching such policy and regulations;
- (iv) I confirm that I have declared in the report about the usage of AI and other generative models, including but not limited to ChatGPT, LLaMA, Gemini, Mistral, and Stable Diffusion, and complied with the instructions provided by HKMU; and
- (v) I am aware that I should be held responsible and liable to disciplinary actions, irrespective of whether I have signed the declaration and whether I have contributed, directly or indirectly, to the problematic contents.

I confirm that I have read through and understood the project requirements. I understand that failure to comply with the project requirements will result in score deduction.

NAME	SID
NGAN Chi Ho	11552520

2. Each student must acknowledge ALL external resources or code from others referenced. We will only mark the parts that are original, not the parts from others.

1. Introduction: This project presents the design and implementation of a “Music Lesson Management System” which intended for private music teachers. The motivation comes from my real-life experience. One day, I arrived at my private music teacher’s studio for a markup lesson, however, he forgot my lesson. He relied on memory and informal WhatsApp message to manage lesson appointments. As a result, the lesson could not take place as planned. 1.1 Problem: This situation are not only inconveniences, they can affect teaching efficiency, student satisfaction, financial tracking, and overall professionalism. A private tutor often performs several roles simultaneously, including instructor, planner, and accountant. Without a dedicated system, important information may be scattered across chat histories, handwritten notes, and payment records. Consequently, they can easily lead to missed appointments, duplicated bookings, weak financial record-keeping, or confusion when lessons are rescheduled. These issues are frequently discussed in online forums with same issue, where common solutions involve the use of commercial management systems or mobile applications, which may be costly or overly complex for individual tutors. 1.2 Solution: The aim of this project is to develop a lightweight but practical software system for managing students, lesson schedules, lesson changes, time conflict checking, invoice generation, and monthly income reporting. 2. Overview: This system starts with a simple login system in the CLI by running “main.py”, where the user enters the default account and password to access the program. After login, user may choose in GUI or CLI mode. The CLI is useful for direct text input and basic operations, while the GUI provides a more user-friendly and visual interface. In GUI mode, all student records are displayed in a table showing key information such as name, instrument, grade, lesson mode, lesson day, start time, lesson duration, fee, and place. This allows users to view and manage all data in one screen more efficiently instead of reading line by line from terminal output. The system supports core functions including adding, editing, and deleting students, generating invoices, exporting monthly income statements, and managing lessons such as cancellation, rescheduling, and conflict detection. In addition, the monthly schedule panel allows users to view the full timetable or check which students have lessons on a specific date. The system is divided into multiple modules. “models.py” defines the core domain entities, “repository.py” handles persistent storage, “services.py” coordinates student-related operations, “schedule_service.py” manages scheduling logic, “invoice.py” generates invoice, “validators.py” centralizes input validation, and the GUI and CLI files provide the user interface. The application is “bootstrapped” through a factory function that wires together the “repository”, “student service”, and “invoice”. 2.1. Core Function in Schedule Generation: For regular students, the system generates lesson dates in a month by following a fixed weekly pattern. For flexible students, the system reads explicitly stored custom lesson records and selects those that

fall within the requested month. These two behaviors are encapsulated in separate lesson plan classes, while the scheduling service decides which strategy to use in “schedule_service.py” that “get_plan()” returns either “RegularLessonPlan()” or “FlexibleLessonPlan()”. Depending on whether the student is regular or flexible. Both plan in “class LessonPlan()” and implement the same abstract interface in “lesson_plan.py”.

2.2 Lesson Change: A recurring weekly schedule is not fully realistic. For this reason, the project introduces lesson overrides. A lesson override stores a change to an originally scheduled regular lesson that may be changed, cancelled, or moved. Through “lesson_overrides”, which is stored inside the “Student” object as a dictionary. Each override maps an original regular lesson date to a changed lesson record. “RegularLessonPlan” uses this dictionary to determine whether a scheduled lesson should remain unchanged, appear as cancelled, or be moved to another date and time. The GUI includes buttons for “Change Selected Lesson,” “Cancel Selected Lesson,”

2.3 Conflict detection and validation: The system does not merely store data, it actively checks whether schedules are valid. The validation layer is centralized in “InputValidator”, which checks date format, time format, weekday, place, and lesson duration. On top of this, “validate_student_schedule()” checks whether a regular student’s weekly lesson conflicts with an existing timetable and whether a flexible student contains duplicate or overlapping lessons. “check_single_lesson_conflict()” further verifies that a changed lesson does not overlap with another concrete lesson on the same date.

2.4 Schedule Presentation: “schedule_for_month()” produce a full monthly schedule, allowing the user to inspect all active lessons within a specified month. Second, it can display which students have lessons on a specific date. The selected student preview panel provides a detailed view of the chosen student’s schedule for the current month which including any changed or cancelled lessons. It also supports sorting by different columns.

2.5 Invoice and Statement: An “Invoice” object for a selected student and month by collecting all active lesson and calculating the total fee accordingly. The “InvoiceService” then exports in PDF. In addition, the system can produce a monthly income statement, giving the teacher a clearer earnings markdown.

3. Object Oriented programming: Encapsulation is demonstrated by grouping related data and operations within the same classes. The “Student” class stores all important student information in one structured object, including identity, lesson details, scheduling mode, overrides, and custom lessons. Likewise, “LessonOccurrence” encapsulates the details of an actual lesson session, and “Invoice” encapsulates billing-related data.

3.1 Abstraction: It is visible in both the domain and persistence layers. In “repository.py”, the abstract class “StudentRepository” defines “list_students()”, “get_by_id()”, “add()”, “update()”, “delete_by_id()”, and “next_id()” without fixing the storage mechanism. This allows the concrete “JsonStudentRepository” class to

implement the details while preserving a clear abstraction boundary. 3.2 Inheritance: It appears in the relationship between “Student” and “Person” in “models.py”, where “Student” provides the concrete implementation of “display_name()”. It also appears in the lesson planning structure, where multiple lesson-plan classes follow a common abstract lesson-plan interface. 3.3 Polymorphism: It is demonstrated by “get_plan()” in “schedule_service.py”. This method may return either a regular or flexible lesson-plan object, after which the program simply calls get_occurrences() on the result. The calling code does not need to know which subclass was selected, because both share the same interface. 3.4: Composition: In “bootstrap.py”, it integrates repository, service, and invoice components using dependency injection and composition, ensuring a loosely coupled and scalable system architecture. 3.5 Class Methods: The “from_dict()” in models.py uses @classmethod and reconstructs a “Student” instance from JSON dictionary data by returning. This is an appropriate design because the logic for rebuilding student objects. 3.6 Static Methods: the “InputValidator” utility in “validators.py”, which is referenced throughout the codebase from “cli.py”, “gui.py”, “services.py” and so on. Methods such as validate_name() do not depend on instance state for validation centralized. 3.7 Magic Methods: It mainly use in “gui.py” for display. 3.8 Class attributes: For defines most of the shared value like name, instrument, grade in class “Student” with different data type. 5. Library: Python built-in libraries such as “json” for data storage and retrieval, “datetime” for handling date and time operations, “pathlib” for file path management, and “Tkinter” for developing the graphical user interface, as well as a third-party library, “ReportLab”, which is used for generating PDF documents such as invoices and income statements. 6. Test case: Default regular student saves in JSON and manual added in the student list. Suppose Student A already has a lesson on Monday at 1300. If the user attempts to add Student B with the same date and time. It will reject the operation and raise a conflict error with overlapping time. The expected result is no record being added. When the lesson changes to another date. The expected result is that the selected lesson appears with [CHANGED] in the preview and the cancelled lesson which appears with [CANCELLED]. The calculations in lesson will not include it. 7. Weakness: The system uses a JSON repository, which is simple but inefficient for large data and concurrent use. It can be improved by using a database such as SQLite. Second, the authentication is default, it should be replaced with secure user login system. Also, it lacks a notification feature for reminders or schedule updates. All idea can be added for future improvement and expand to more similar self-employee like private teacher, sport instructor. 8 Conclusion: It addresses the key difficulties faced by private music teachers by combining student records, automated schedule, conflict detection, flexible-lesson handling, lesson override management, invoice generation, and income reporting.

Reference:

Python Student Management System – Simplify Your School Operations:

<https://pythongeeks.org/python-student-management-system/>

Student Results Management System Using Tkinter

<https://www.geeksforgeeks.org/python/student-results-management-system-using-tkinter/>

Appendix

GitHub link:

<https://github.com/johnngan0511/11552520>

1.Real forum discussion and solution

https://www.reddit.com/r/MusicTeachers/comments/1pj4xwr/music_teachers_what_software_do_you_use_to_manage/

https://www.reddit.com/r/smallbusiness/comments/1q4yxp4/software_recommendations_for_scheduling_in_music/?utm_source=chatgpt.com

2.Login In AC PW

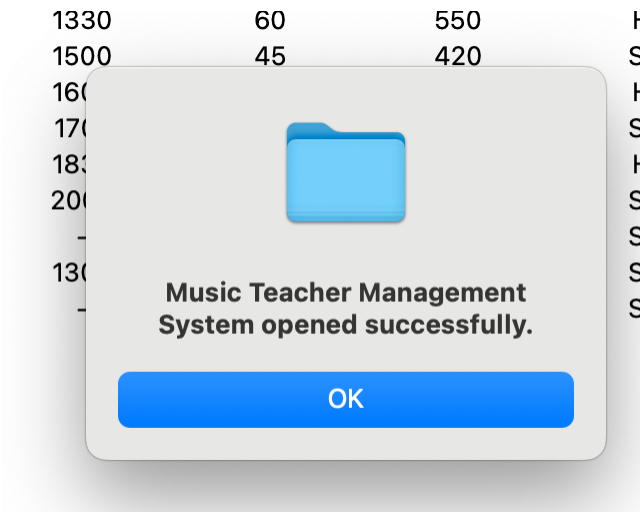
```
===== Login System =====  
Account (default 8090): 8090  
Password (default 8090): 8090  
Login success.  
  
Open GUI now? (y/n): █
```

2. CLI Manu

```
=== Music Teacher Management ===
1) View my students
2) Add student (regular / flexible)
3) Edit student
4) Delete student
5) Show my schedule (one month)
6) Generate invoice (PDF)
7) Add flexible lesson
8) Remove flexible lesson
0) Exit
Choose: 0
Bye!!!
[SYSTEM] main() finished
```

```
Choose: 1
=== My Students ===
ID=1 | Name=John Chan | Birth=2001-01-01 | Instrument=Trumpet | Grade=8 | Mode=Regular | Start=9900 | Time=45min | Fee=HKD500 | Lesson=Mon | Place=Studio
ID=2 | Name=Alan Lee | Birth=2002-03-12 | Instrument=Piano | Grade=6 | Mode=Regular | Start=1000 | Time=45min | Fee=HKD450 | Lesson=Tue | Place=Studio
ID=3 | Name=Jace Wong | Birth=2003-07-20 | Instrument=Violin | Grade=5 | Mode=Regular | Start=1100 | Time=45min | Fee=HKD400 | Lesson=Wed | Place=Home
ID=4 | Name=Mary Ho | Birth=2000-11-02 | Instrument=Flute | Grade=7 | Mode=Regular | Start=1200 | Time=45min | Fee=HKD480 | Lesson=Thu | Place=Studio
ID=5 | Name=Chris Lau | Birth=2004-09-14 | Instrument=Drums | Grade=4 | Mode=Regular | Start=1330 | Time=60min | Fee=HKD550 | Lesson=Fri | Place=Home
ID=6 | Name=Emily Ng | Birth=2005-05-10 | Instrument=Guitar | Grade=6 | Mode=Regular | Start=1500 | Time=45min | Fee=HKD420 | Lesson=Sat | Place=Studio
ID=7 | Name=David Chan | Birth=2002-12-25 | Instrument=Saxophone | Grade=7 | Mode=Regular | Start=1600 | Time=45min | Fee=HKD500 | Lesson=Sun | Place=Home
ID=8 | Name=Sophia Lam | Birth=2003-08-18 | Instrument=Cello | Grade=5 | Mode=Regular | Start=1700 | Time=60min | Fee=HKD530 | Lesson=Mon | Place=Studio
ID=9 | Name=Daniel Cheung | Birth=2001-04-30 | Instrument=Clarinet | Grade=6 | Mode=Regular | Start=1830 | Time=45min | Fee=HKD460 | Lesson=Tue | Place=Home
ID=10 | Name=Kevin Yeung | Birth=2004-06-22 | Instrument=Trombone | Grade=5 | Mode=Regular | Start=2000 | Time=45min | Fee=HKD470 | Lesson=Wed | Place=Studio
ID=11 | Name=Jason Lee | Birth=2001-01-01 | Instrument=Trumpet | Grade=5 | Mode=Flexible | Start=- | Time=- | Fee=HKD450 | Lesson=- | Place=Studio
ID=12 | Name=Jason Wong | Birth=2001-01-01 | Instrument=Trumpet | Grade=6 | Mode=Regular | Start=1300 | Time=45min | Fee=HKD500 | Lesson=Wed | Place=Studio
ID=13 | Name=Peter Lee | Birth=2001-01-01 | Instrument=Trumpet | Grade=5 | Mode=Flexible | Start=- | Time=- | Fee=HKD500 | Lesson=- | Place=Studio
```

2.GUI interface open



2.GUI interface

Music Teacher Management System

Month (YYYY-MM): 2026-02

Refresh

Add Student

Edit Student

Delete Student

Generate Invoice

Export Income Statement

ID	Name	Instrument	Grade	Mode	Lesson Day	Start Time	Minutes	Fee(HKD)	Place
1	John Chan	Trumpet	8	Regular	Mon	0900	45	500	Studio
2	Alan Lee	Piano	6	Regular	Tue	1000	45	450	Studio
3	Jace Wong	Violin	5	Regular	Wed	1100	45	400	Home
4	Mary Ho	Flute	7	Regular	Thu	1200	45	480	Studio
5	Chris Lau	Drums	4	Regular	Fri	1330	60	550	Home
6	Emily Ng	Guitar	6	Regular	Sat	1500	45	420	Studio
7	David Chan	Saxophone	7	Regular	Sun	1600	45	500	Home
8	Sophia Lam	Cello	5	Regular	Mon	1700	60	530	Studio
9	Daniel Cheung	Clarinet	6	Regular	Tue	1830	45	460	Home
10	Kevin Yeung	Trombone	5	Regular	Wed	2000	45	470	Studio
11	Jason Lee	Trumpet	5	Flexible	-	-	-	450	Studio

Monthly Schedule

2026-02-01 - Sun - 1600-1645 (45min) - David Chan

2026-02-02 - Mon - 0900-0945 (45min) - John Chan

2026-02-02 - Mon - 1700-1800 (60min) - Sophia Lam

2026-02-03 - Tue - 1000-1045 (45min) - Alan Lee

2026-02-03 - Tue - 1830-1915 (45min) - Daniel Cheung

2026-02-04 - Wed - 1100-1145 (45min) - Jace Wong

2026-02-04 - Wed - 2000-2045 (45min) - Kevin Yeung

2026-02-05 - Thu - 1200-1245 (45min) - Mary Ho

2026-02-06 - Fri - 1330-1430 (60min) - Chris Lau

2026-02-07 - Sat - 1500-1545 (45min) - Emily Ng

2026-02-08 - Sun - 1600-1645 (45min) - David Chan

2026-02-09 - Mon - 0900-0945 (45min) - John Chan

2026-02-09 - Mon - 1700-1800 (60min) - Sophia Lam

2026-02-10 - Tue - 1000-1045 (45min) - Alan Lee

2026-02-10 - Tue - 1830-1915 (45min) - Daniel Cheung

2026-02-11 - Wed - 1100-1145 (45min) - Jace Wong

2026-02-11 - Wed - 2000-2045 (45min) - Kevin Yeung

2026-02-12 - Thu - 1200-1245 (45min) - Mary Ho

2026-02-13 - Fri - 1330-1430 (60min) - Chris Lau

2026-02-14 - Sat - 1500-1545 (45min) - Emily Ng

2026-02-15 - Sun - 1600-1645 (45min) - David Chan

2026-02-16 - Mon - 0900-0945 (45min) - John Chan

2026-02-16 - Mon - 1700-1800 (60min) - Sophia Lam

2026-02-17 - Tue - 1000-1045 (45min) - Alan Lee

2026-02-17 - Tue - 1830-1915 (45min) - Daniel Cheung

2026-02-18 - Wed - 1100-1145 (45min) - Jace Wong

2026-02-18 - Wed - 2000-2045 (45min) - Kevin Yeung

2026-02-19 - Thu - 1200-1245 (45min) - Mary Ho

2026-02-19 - Thu - 1300-1345 (45min) - Jason Lee

2026-02-20 - Fri - 1330-1430 (60min) - Chris Lau

2026-02-21 - Sat - 1500-1545 (45min) - Emily Ng

2026-02-22 - Sun - 1600-1645 (45min) - David Chan

2026-02-23 - Mon - 0900-0945 (45min) - John Chan

2026-02-23 - Mon - 1700-1800 (60min) - Sophia Lam

2026-02-24 - Tue - 1000-1045 (45min) - Alan Lee

2026-02-24 - Tue - 1830-1915 (45min) - Daniel Cheung

2026-02-25 - Wed - 1100-1145 (45min) - Jace Wong

2026-02-25 - Wed - 2000-2045 (45min) - Kevin Yeung

2026-02-26 - Thu - 1200-1245 (45min) - Mary Ho

2026-02-27 - Fri - 1330-1430 (60min) - Chris Lau

2026-02-28 - Sat - 1500-1545 (45min) - Emily Ng

Date (YYYY-MM-DD): 2026-02-01

Show Students On Date

Jason Lee | Trumpet | Flexible | 2026-02 | 1 lesson(s)

Lesson 1: 2026-02-19 - Thu - 1300-1345 (45min)

Flexible student selected. Change / Cancel / Clear are only for regular mode.

Change Selected Lesson

Cancel Selected Lesson

Clear Selected Override

Music Teacher Management System

Month (YYYY-MM): 2026-02

Refresh

Add Student

Edit Student

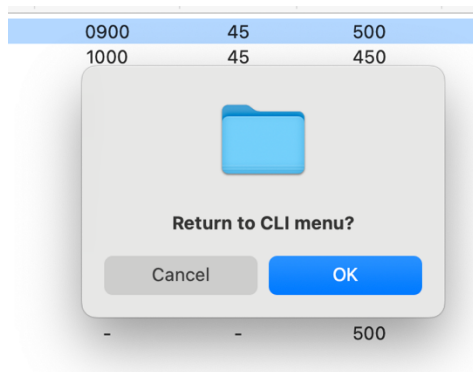
Delete Student

Generate Invoice

Export Income Statement

ID	Name	Instrument	Grade	Mode	Lesson Day	Start Time	Minutes	Fee(HKD)	Place
1	John Chan	Trumpet	8	Regular	Mon	0900	45	500	Studio
2	Alan Lee	Piano	6	Regular	Tue	1000	45	450	Studio
3	Jace Wong	Violin	5	Regular	Wed	1100	45	400	Home
4	Mary Ho	Flute	7	Regular	Thu	1200	45	480	Studio
5	Chris Lau	Drums	4	Regular	Fri	1330	60	550	Home
6	Emily Ng	Guitar	6	Regular	Sat	1500	45	420	Studio
7	David Chan	Saxophone	7	Regular	Sun	1600	45	500	Home
8	Sophia Lam	Cello	5	Regular	Mon	1700	60	530	Studio
9	Daniel Cheung	Clarinet	6	Regular	Tue	1830	45	460	Home
10	Kevin Yeung	Trombone	5	Regular	Wed	2000	45	470	Studio
11	Jason Lee	Trumpet	5	Flexible	-	-	-	450	Studio

2. GUI close



```
=== Music Teacher Management ===
1) View my students
2) Add student (regular / flexible)
3) Edit student
4) Delete student
5) Show my schedule (one month)
6) Generate invoice (PDF)
7) Add flexible lesson
8) Remove flexible lesson
0) Exit
Choose: 0
Bye!!!
[SYSTEM] main() finished
```

2.1 Student add in Regular

Add Student

Name: Jason Wong

Birth: 2001-01-01

Instrument: Trumpet

Grade: 6

Lesson minutes: 45

Start time: 1300

Fee (HKD): 500

Lesson weekday: Wed

Place: Studio

☐ Flexible mode (non-regular)

Regular mode: fixed weekly lesson schedule.

Save Cancel

Music Teacher Management System

Month (YYYY-MM): 2026-02 Refresh Add Student Edit Student Delete Student Generate Invoice

ID	Name	Instrument	Grade	Mode	Lesson Day	Start Time	Minutes	Fee(HK
1	Peter Lee	Trumpet	5				45	500
2							45	450
3							45	400
4							45	480
5							60	550
6							45	420
7							45	500
8							60	530
9							45	460
10							45	470
11							45	450
12							45	500

Edit Student

Name: Peter Lee

Birth: 2001-01-01

Instrument: Trumpet

Grade: 5

Lesson minutes: 45

Start time: 1300

Fee (HKD): 500

Lesson weekday: Mon

Place: Studio

☒ Flexible mode (non-regular)

Flexible mode: weekly day/time fields are disabled. Please add custom lessons below.

Flexible Lessons

Lesson	Date	Day	Time	Duration
Lesson 1:	2026-02-11	Wed	1400-1445	(45min)

Add Flexible Lesson

Date (YYYY-MM-DD): 2026-02-15

Start time (HHMM): 1400

Lesson minutes: 45

Example: 2026-02-18 / 1400 / 45

Save Cancel

Peter Lee

Lesson

Add Lesson Edit Selected Remove Selected

Save Cancel

2.2 Delete Student

Month (YYYY-MM): 2026-02

Refresh

Add Student

Edit Student

Delete Student

Gen

ID	Name	Instrument	Grade	Mode	Lesson Day	Start Time	Minutes
1	John Chan	Trumpet	8	Regular	Mon	0900	45
2	Alan Lee	Piano	6	Regular	Tue	1000	45
3	Jace Wong	Violin	5	Regular	Wed	1100	45
4	Mary Ho	Flute	7	Regular	Thu	1200	45
5	Chris Lau	Drums	4	Regular	Fri	1330	60
6	Emily Ng	Guitar	6	Regular	Sat	1500	45
7	David Chan	Saxophone	7			00	45
8	Sophia Lam	Cello	5			00	60
9	Daniel Cheung	Clarinet	6			00	45
10	Kevin Yeung	Trombone	5			00	45
11	Jason Lee	Trumpet	5				-
12	Jason Wong	Trumpet	6			00	45
13	Peter Lee	Trumpet	5				-

Delete student ID=12?

No

Yes

2.2 Lesson changed, cancelled and button

Change Selected Lesson

Original lesson:

2026-02-05 - Thu - 2100-2145 (45min)
[CHANGED]

New date (YYYY-MM-DD):

2026-02-05

New start time (HHMM):

2100

Save

Cancel

Kevin Yeung | Trombone | Regular | 2026-02 | 4 lesson(s)

Lesson 1: 2026-02-05 - Thu - 2100-2145 (45min) [CHANGED]

Lesson 2: 2026-02-11 - Wed - 2000-2045 (45min)

Lesson 3: 2026-02-18 - Wed - 2000-2045 (45min)

Lesson 4: 2026-02-25 - Wed - 2000-2045 (45min)

Selected original lesson date: 2026-02-04

Change Selected Lesson

Cancel Selected Lesson

Clear Selected Override



**Selected lesson
cancelled successfully.**

OK

Kevin Yeung | Trombone | Regular | 2026-02 | 4 lesson(s)

Lesson 1:	2026-02-05	-	Thu	-	2100-2145	(45min)	[CHANGED]
Lesson 2:	2026-02-11	-	Wed	-	2000-2045	(45min)	[CANCELLED]
Lesson 3:	2026-02-18	-	Wed	-	2000-2045	(45min)	
Lesson 4:	2026-02-25	-	Wed	-	2000-2045	(45min)	

Select one lesson below, then click Change / Cancel.

Change Selected Lesson

Cancel Selected Lesson

Clear Selected Override

2.3 Overlap



Time overlap detected.

Existing student: Mary Ho
Date: 2026-02-19
Time: 1200-1245 (45min)

OK

Kevin Yeung | Trombone | Regular | 2026-02 | 4 lesson(s)

Lesson 1:	2026-02-05	-	Thu	-	2100-2145	(45min)	[CHANGED]
Lesson 2:	2026-02-11	-	Wed	-	2000-2045	(45min)	[CANCELLED]
Lesson 3:	2026-02-18	-	Wed	-	2000-2045	(45min)	
Lesson 4:	2026-02-25	-	Wed	-	2000-2045	(45min)	

2.4 Schedule In Day

Students on 2026-02-02
=====

2026-02-02 - Mon - 0900-0945 (45min) - John Chan - Trumpet - Studio
2026-02-02 - Mon - 1700-1800 (60min) - Sophia Lam - Cello - Studio



Date (YYYY-MM-DD):

Show Students On Date

2.4 Schedule In Month

Monthly Schedule

2026-02-01 - Sun - 1600-1645 (45min) - David Chan
2026-02-02 - Mon - 0900-0945 (45min) - John Chan
2026-02-02 - Mon - 1700-1800 (60min) - Sophia Lam
2026-02-03 - Tue - 1000-1045 (45min) - Alan Lee
2026-02-03 - Tue - 1830-1915 (45min) - Daniel Cheung
2026-02-04 - Wed - 1100-1145 (45min) - Jace Wong
2026-02-04 - Wed - 1300-1345 (45min) - Jason Wong
2026-02-05 - Thu - 1200-1245 (45min) - Mary Ho
2026-02-05 - Thu - 2100-2145 (45min) - Kevin Yeung
2026-02-06 - Fri - 1330-1430 (60min) - Chris Lau
2026-02-07 - Sat - 1500-1545 (45min) - Emily Ng
2026-02-08 - Sun - 1600-1645 (45min) - David Chan
2026-02-09 - Mon - 0900-0945 (45min) - John Chan
2026-02-09 - Mon - 1700-1800 (60min) - Sophia Lam
2026-02-10 - Tue - 1000-1045 (45min) - Alan Lee
2026-02-10 - Tue - 1830-1915 (45min) - Daniel Cheung
2026-02-11 - Wed - 1100-1145 (45min) - Jace Wong
2026-02-11 - Wed - 1300-1345 (45min) - Jason Wong
2026-02-11 - Wed - 1400-1445 (45min) - Peter Lee
2026-02-12 - Thu - 1200-1245 (45min) - Mary Ho
2026-02-13 - Fri - 1330-1430 (60min) - Chris Lau
2026-02-14 - Sat - 1500-1545 (45min) - Emily Ng
2026-02-15 - Sun - 1400-1445 (45min) - Peter Lee
2026-02-15 - Sun - 1600-1645 (45min) - David Chan
2026-02-16 - Mon - 0900-0945 (45min) - John Chan
2026-02-16 - Mon - 1700-1800 (60min) - Sophia Lam
2026-02-17 - Tue - 1000-1045 (45min) - Alan Lee
2026-02-17 - Tue - 1830-1915 (45min) - Daniel Cheung
2026-02-18 - Wed - 1100-1145 (45min) - Jace Wong
2026-02-18 - Wed - 1300-1345 (45min) - Jason Wong
2026-02-18 - Wed - 2000-2045 (45min) - Kevin Yeung
2026-02-19 - Thu - 1200-1245 (45min) - Mary Ho
2026-02-19 - Thu - 1300-1345 (45min) - Jason Lee
2026-02-20 - Fri - 1330-1430 (60min) - Chris Lau
2026-02-21 - Sat - 1500-1545 (45min) - Emily Ng
2026-02-22 - Sun - 1600-1645 (45min) - David Chan
2026-02-23 - Mon - 0900-0945 (45min) - John Chan
2026-02-23 - Mon - 1700-1800 (60min) - Sophia Lam
2026-02-24 - Tue - 1000-1045 (45min) - Alan Lee
2026-02-24 - Tue - 1830-1915 (45min) - Daniel Cheung
2026-02-25 - Wed - 1100-1145 (45min) - Jace Wong
2026-02-25 - Wed - 1300-1345 (45min) - Jason Wong
2026-02-25 - Wed - 2000-2045 (45min) - Kevin Yeung

Date (YYYY-MM-DD):

Show Students On Date

2.5 Invoice

Music Teacher Management System

Month (YYYY-MM): 2026-02

Refresh

Add Student

Edit Student

Delete Student

Generate Invoice

Export Inc

ID	Name	Instrument	Grade	Mode	Lesson Day	Start Time	Minutes	Fee(HKD)	
1	John Chan	Trumpet	8	Regular	Mon	0900	45	500	\$
2	Alan Lee	Piano	6	Regular	Tue	1000	45	450	\$
3	Jace Wong	Violin	5	Regular	Wed				\$
4	Mary Ho	Flute	7	Regular	Thu				\$
5	Chris Lau	Drums	4	Regular	Fri				\$
6	Emily Ng	Guitar	6	Regular	Sat				\$
7	David Chan	Saxophone	7	Regular	Sun				\$
8	Sophia Lam	Cello	5	Regular	Mon				\$
9	Daniel Cheung	Clarinet	6	Regular	Tue				\$
10	Kevin Yeung	Trombone	5	Regular	Wed				\$
11	Jason Lee	Trumpet	5	Flexible	-				\$
12	Jason Wong	Trumpet	6	Regular	Wed				\$
13	Peter Lee	Trumpet	5	Flexible	-				\$

☆ invoice_202602-11-000... × + 建立

編輯 轉換 電子簽署

Music Lesson Invoice

Invoice No.: 202602-11-0001

Month: 2026-02

Student: Jason Lee

Instrument: Trumpet Grade: 5

Lesson Details:

- 2026-02-19 - 1300-1345 (45min)

Fee per lesson (HKD): 450

Total (HKD): 450

Music Teacher Management System

Month (YYYY-MM): 2026-02

Refresh

Add Student

Edit Student

Delete Student

Generate Invoice

Export Inc

ID	Name	Instrument	Grade	Mode	Lesson Day	Start Time	Minutes	Fee(HKD)	Place
1	John Chan	Trumpet	8	Regular	Mon	0900	45	500	Studi
2	Alan Lee	Piano	6	Regular	Tue	1000	45	450	Studi
3	Jace Wong	Violin	5	Regular	Wed				Hom
4	Mary Ho	Flute	7	Regular	Thu				Studi
5	Chris Lau	Drums	4	Regular	Fri				Hom
6	Emily Ng	Guitar	6	Regular	Sat				Studi
7	David Chan	Saxophone	7	Regular	Sun				Hom
8	Sophia Lam	Cello	5	Regular	Mon				Studi
9	Daniel Cheung	Clarinet	6	Regular	Tue				Hom
10	Kevin Yeung	Trombone	5	Regular	Wed				Studi
11	Jason Lee	Trumpet	5	Flexible	-				Studi
12	Jason Wong	Trumpet	6	Regular	Wed				Studi
13	Peter Lee	Trumpet	5	Flexible	-				Studi

Invoice generated:
output/
invoice_202602-1-0001.pdf

☆ invoice_202602-1-0001... ×

+ 建立

編輯 轉換 電子簽署

Music Lesson Invoice

Invoice No.: 202602-1-0001
Month: 2026-02
Student: John Chan
Instrument: Trumpet Grade: 8

- Lesson Details:
- 2026-02-02 - 0900-0945 (45min)
 - 2026-02-09 - 0900-0945 (45min)
 - 2026-02-16 - 0900-0945 (45min)
 - 2026-02-23 - 0900-0945 (45min)

Fee per lesson (HKD): 500
Total (HKD): 2000

2.5 Income statement

Music Teacher Management System

Month (YYYY-MM): 2026-02 Refresh Add Student Edit Student Delete Student Generate Invoice Export Income Statement

ID	Name	Instrument	Grade	Mode	Lesson Day	Start Time	Minutes	Fee(HKD)	Place	Mc
1	John Chan	Trumpet	8	Regular	Mon	0900	45	500	Studio	20
2	Alan Lee	Piano	6	Regular	Tue				Studio	20
3	Jace Wong	Violin	5	Regular	Wed				Home	20
4	Mary Ho	Flute	7	Regular	Thu				Studio	20
5	Chris Lau	Drums	4	Regular	Fri				Home	20
6	Emily Ng	Guitar	6	Regular	Sat				Studio	20
7	David Chan	Saxophone	7	Regular	Sun				Home	20
8	Sophia Lam	Cello	5	Regular	Mon				Studio	20
9	Daniel Cheung	Clarinet	6	Regular	Tue				Home	20
10	Kevin Yeung	Trombone	5	Regular	Wed				Studio	20
11	Jason Lee	Trumpet	5	Flexible	-				Studio	20
12	Jason Wong	Trumpet	6	Regular	Wed				Studio	20
13	Peter Lee	Trumpet	5	Flexible	-				Studio	20

Monthly income statement generated: output/monthly_income_2026-02.pdf
OK

Monthly Income Statement

Month: 2026-02

ID	Name	Instrument	Grade	Lessons	Fee	Total
1	John Chan	Trumpet	8	4	HKD 500	HKD 2000
2	Alan Lee	Piano	6	4	HKD 450	HKD 1800
3	Jace Wong	Violin	5	4	HKD 400	HKD 1600
4	Mary Ho	Flute	7	4	HKD 480	HKD 1920
5	Chris Lau	Drums	4	4	HKD 550	HKD 2200
6	Emily Ng	Guitar	6	4	HKD 420	HKD 1680
7	David Chan	Saxophone	7	4	HKD 500	HKD 2000
8	Sophia Lam	Cello	5	4	HKD 530	HKD 2120
9	Daniel Cheung	Clarinet	6	4	HKD 460	HKD 1840
10	Kevin Yeung	Trombone	5	3	HKD 470	HKD 1410
11	Jason Lee	Trumpet	5	1	HKD 450	HKD 450
12	Jason Wong	Trumpet	6	4	HKD 500	HKD 2000
13	Peter Lee	Trumpet	5	2	HKD 500	HKD 1000

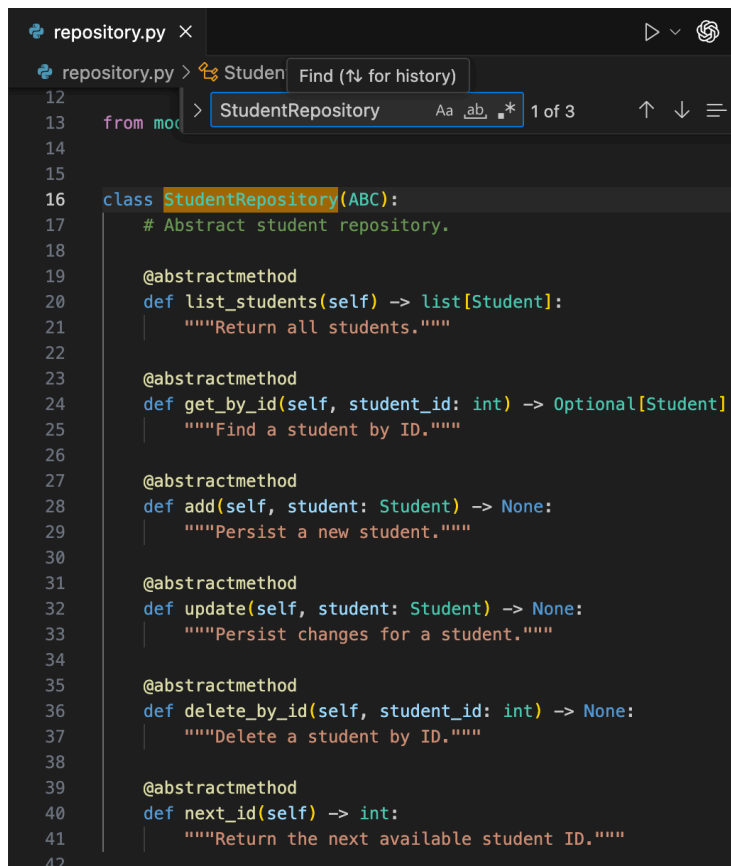
Total Monthly Income: HKD 22020

3. Encapsulation

```

models.py ×
models.py > LessonOccurrence
21         """Return a human-readable n
22
23
24 @dataclass
25 class LessonOccurrence:
26     """A concrete lesson occurrence for a calendar date."""
27
28     original_date: str
29     date: str
30     weekday: str
31     start_time: str
32     end_time: str
33     lesson_minutes: int
34     cancelled: bool = False
35     is_override: bool = False
  
```

3.1 Abstraction

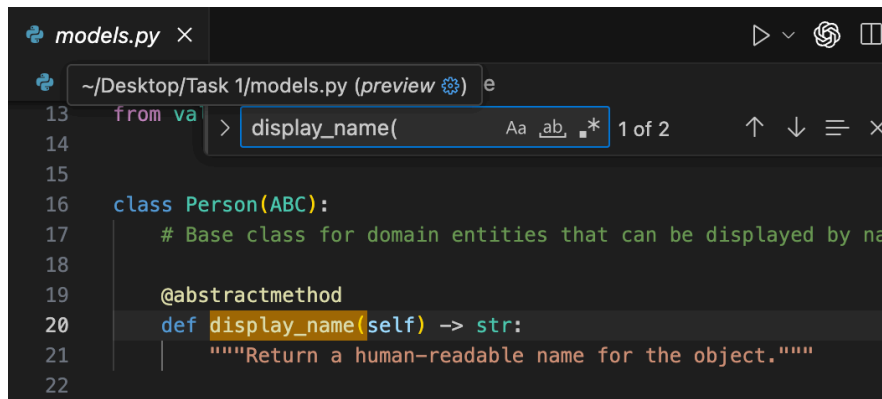


```

12
13 from mod > StudentRepository Aa .ab .* 1 of 3
14
15
16 class StudentRepository(ABC):
17     # Abstract student repository.
18
19     @abstractmethod
20     def list_students(self) -> list[Student]:
21         """Return all students."""
22
23     @abstractmethod
24     def get_by_id(self, student_id: int) -> Optional[Student]:
25         """Find a student by ID."""
26
27     @abstractmethod
28     def add(self, student: Student) -> None:
29         """Persist a new student."""
30
31     @abstractmethod
32     def update(self, student: Student) -> None:
33         """Persist changes for a student."""
34
35     @abstractmethod
36     def delete_by_id(self, student_id: int) -> None:
37         """Delete a student by ID."""
38
39     @abstractmethod
40     def next_id(self) -> int:
41         """Return the next available student ID."""
42

```

3.2 Inheritance



```

13 from va > display_name( Aa .ab .* 1 of 2
14
15
16 class Person(ABC):
17     # Base class for domain entities that can be displayed by na
18
19     @abstractmethod
20     def display_name(self) -> str:
21         """Return a human-readable name for the object."""
22

```


3.3 Polymorphism

```

schedule_service.py > ScheduleService > lesson_dates_in_month
356         return {"month": month, "plan": self.get_plan(student)}
41     def build_occurrence(self, occurrence: dict) -> LessonPlan:
64
65
66     def get_plan(self, student: Student) -> LessonPlan:
67         # Return the appropriate lesson plan strategy for a student.
68         return FlexibleLessonPlan() if student.is_flexible else RegularLessonPlan()

schedule_service.py > ScheduleService > _check_regular_conflict
70     return {"month": f"{year}-{month:02d}", "rows": statement_rows, "total_income": total_income}
71     def lesson_dates_in_month(self, year: int, month: int, weekday: str) -> list[str]:
81         current_date = datetime(year, month, 1)
82         return lesson_dates
83
84     def get_student_occurrences(self, student_id: int, year: int, month: int) -> list[dict]:
85         # Return occurrence dictionaries for the target student and month.
86         student = self.repo.get_by_id(InputValidator.validate_student_id(student_id))
87         if not student:
88             raise ValueError("Student not found.")
89         return [occurrence.to_dict() for occurrence in self.get_plan(student).get_occurrences(self, student, year, month)]

```

3.4: Composition

```

bootstrap.py > create_student_service
1  # =====
2  # Factory helpers to bootstrap the application
3  # =====
4  from __future__ import annotations # Allow forward references in type hints
5  from constants import DATA_PATH # Default path for storing student data
6  from invoice import InvoiceService # Service responsible for invoice generation
7  from repository import JsonStudentRepository # Data access layer
8  from services import StudentService # Core logic layer
9
10 def create_student_service(data_path: str = DATA_PATH) -> StudentService:
11     # Create the fully wired student service.
12
13     # Step 1: Create repository (Data Layer)
14     # This handles all persistence (loading/saving students from JSON file)
15     repo = JsonStudentRepository(data_path)
16

```

3.5 Class Methods

```

models.py > ...
138     }
139     def to_dict(self) -> dict[str, Any]:
140         return {
141             "id": self.id,
142             "name": self.name,
143             "birth_date": self.birth_date.isoformat(),
144             "instrument": self.instrument,
145             "grade": self.grade,
146             "lesson_minutes": self.lesson_minutes,
147             "lesson_start_time": self.lesson_start_time,
148             "fee_hkd": self.fee_hkd,
149             "lesson_weekday": self.lesson_weekday,
150             "place": self.place,
151             "plan_type": self.plan_type,
152             "lesson_overrides": self.lesson_overrides,
153             "custom_lessons": self.custom_lessons,
154         }
155
156     @classmethod
157     def from_dict(cls, data: dict[str, Any]) -> "Student":
158         # Build a student from JSON data.
159         birth_date = InputValidator.parse_date(str(data["birth_date"]), "Birth date")
160         lesson_overrides = data.get("lesson_overrides", []) if isinstance(data.get("lesson_overrides", []), dict) else []
161         custom_lessons = data.get("custom_lessons", []) if isinstance(data.get("custom_lessons", []), list) else []
162         plan_type = InputValidator.validate_plan_type(str(data.get("plan_type", "regular")), "regular") if str(data.get("plan_type", "regular")).strip().lower() in {"regular", "flexible"} else "regular"
163         # Validate all fields and construct the Student object.
164         return cls(
165             student_id=InputValidator.validate_student_id(data["student_id"]),
166             name=InputValidator.validate_name(str(data["name"])),
167             birth_date=InputValidator.validate_date(birth_date),
168             instrument=InputValidator.validate_instrument(str(data["instrument"])),
169             grade=InputValidator.validate_grade(str(data.get("grade", ""))),
170             lesson_minutes=InputValidator.validate_lesson_minutes(data.get("lesson_minutes", 45)),
171             lesson_start_time=InputValidator.validate_time_hmm(str(data.get("lesson_start_time", "1300"))),
172             fee_hkd=InputValidator.validate_fee_hkd(data["fee_hkd"]),
173             lesson_weekday=InputValidator.validate_weekday(str(data.get("lesson_weekday", "Mon"))),
174             place=InputValidator.validate_place(str(data["place"])),
175             plan_type=plan_type,
176             lesson_overrides=lesson_overrides,
177             custom_lessons=[InputValidator.validate_flexible_lesson(item, data.get("lesson_minutes", 45)) for item in custom_lessons],
178         )

```

3.6 Static Methods

```

validators.py ×
validators.py > InputValidator > validate_name
12
13 class InputValidator:
14     """Centralized validation to keep UI code small and consistent."""
15
16     @staticmethod
17     def validate_non_empty_text(value: str, field_name: str) -> str:
18         """Validate a required text field."""
19         normalized_value = str(value).strip()
20         if not normalized_value:
21             raise ValidationError(f"{field_name} cannot be empty.")
22         return normalized_value
23

```

3.7 Magic Methods

```

gui.py ×
gui.py > App
14 class App(tk.Tk):
15
16     def __init__(self, student_service):
17         super().__init__()
18         self.service = student_service
19         self.title("Music Teacher Management System")
20         self.geometry("1600x900")
21         self.minsize(1200, 820)
22         self.current_occurrences: list[dict] = []
23         self.columns = ["id", "name", "instrument", "grade", "mode", "weekday"]
24         self.sort_states = {column_name: False for column_name in self.columns}
25         self.weekday_order = {name: index for index, name in enumerate(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"])}
26         self.grade_order = {grade: index for index, grade in enumerate(["1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "11", "12"])}
27         self._build_layout()
28         self.refresh()
29         self.show_schedule()
30         self.after(200, self._show_open_message) # show popup after GUI loads
31         self.protocol("WM_DELETE_WINDOW", self.on_close)
32
33

```